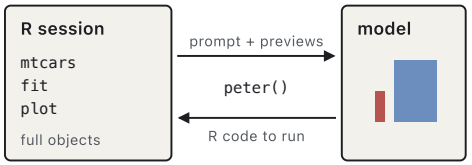
# Agents in your R session with gptr :: CHEATSHEET

`peter()` runs language-model agents on the objects already in your R session. Named for Peter Wason, whose dual-process research inspires its System 1 and System 2 roles, and Peter Naur, of Backus-Naur form: sessions become readable, runnable documents.

## How it works




Objects stay in your R session and the agent's R code runs there on the full data. The model sees the prompt, bounded previews and tool output.

**`gptr_describe`**`(mtcars)`
What a model is shown of an object.

## Set up

`pak::`**`pak`**`("Broccolito/gptr@main")`
**`library`**`(gptr)`
Install gptr 1.0 from GitHub (R 4.2 or later).

**`gptr_env`**`("~/keys/.env")`
Load provider keys from a private .env file; values are registered for redaction.

**`gptr_login`**`("openrouter")`
Sign in interactively: OAuth for OpenRouter, masked key entry for other providers.

**`gptr_providers`**`()`
Which routes are ready: credential, status, default model, egress.

**`gptr_models`**`("sonnet")`
Search the offline model catalog by name, alias or provider.

**`gptr_config`**`(model = opus, .scope = "user")`
Save a default model. Scopes: `"session"`, `"project"`, `"user"`.

**`gptr_init`**`(".")`
**`gptr_trust`**`(".", trust = TRUE)`
Create a `.gptr/` project workspace, then trust its settings.

**`gptr_permissions`**`(allow = "r(fn:summary)")`
Add a permission rule for this R session; no arguments lists the rules.

`fake = `**`gptr_fake_provider`**`(`**`list`**`("32 rows."))`
A scripted offline model for examples and tests: no key, no network.

With no model configured, the first interactive `peter()` offers the Codex CLI, the Claude Code CLI or manual API or Ollama setup.

## Ask Peter from R

Give `peter()` a prompt and the objects to work on; it returns a session. It works in scripts, loops and functions.

`s = `**`peter`**`("Fit mpg on wt; save it as fit.", mtcars)`
The prompt first, then the objects to work on.

`s$text; s$value; s$turns`
Final answer, the value chosen with `gptr_return()`, completed turns.

`s$usage; s$cost; s$envir`
Usage, total cost and the agent's workspace.

```
col = "mpg"; n = 3
peter("Plot {col} for the top {n} cars.")
```
Literal prompts interpolate `{name}` from R.

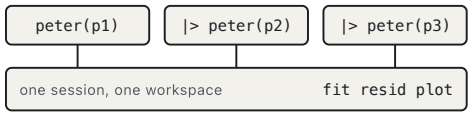
**`peter`**`("Clean the data.", raw, mode = auto, model = "openai/gpt-6-sol")`
Name other arguments exactly; bare names work.

**`peter`**`("Summarise.", sales, budget = `**`list`**`(tokens = 5e4, turns = 5))`
Per-call limits on tokens, cost and turns.

**`gptr_return`**`(`**`coef`**`(fit))` `# in agent R code`
Marks the run's result, read back as `s$value`.

**`tryCatch`**`(`**`peter`**`("Fit.", d), gptr_error = `**`function`**`(e) e$session)`
Errors carry the session as `e$session`.

## Pipe: one session, many turns




`s |> `**`peter`**`("Now plot the residuals.")`
Same session and workspace, one more turn.

**`peter`**`("Draft a summary.", fit) |> `**`peter`**`("Shorten it.")`
Each pipe adds a turn to the session on its left.

```
b = gptr_fork(s)
b |> peter("Try a log model instead.")
```
Branch off; the original stays unchanged.

`s |> `**`peter`**`("Add a caveat.", model = opus)`
Switch the model mid-pipe; the history is kept.

`s; `**`summary`**`(s); `**`gptr_last`**`()`
Print it, its history, the most recent session.

## The console

```
session = peter()
session |> peter()
```
Open the console; `/exit` returns the session.

### AT THE PETER> PROMPT

| | |
|---|---|
| `Fit mpg on wt.` | A prompt for the session |
| `!head(d)` | Run R and show the model |
| `!!x = 2` | Run R without telling the model |
| ```` ```r … ``` ```` | A fenced block runs like `!code` |
| `"""` … `"""` | A multi-line prompt |
| `line \` | Continue on the next line |
| `@notes.md` | Attach a file's first 40 lines |
| `@fit` | Attach an object |
| `{n}` | Insert an R value |

### SLASH COMMANDS

| | |
|---|---|
| `/help [cmd]` | Commands and syntax |
| `/model [ref]` | Show or switch model |
| `/mode [mode]` | Show or switch mode |
| `/status` | Session, model, mode |
| `/cost /context` | Tokens and cost |
| `/env [pattern]` | Workspace objects |
| `/tools` | Tools and helpers |
| `/compact [focus]` | Compact the history |
| `/clear` | New chat, same objects |
| `/resume [id]` | Resume a stored session |
| `/fork` | Fork and continue there |
| `/doc [path]` | The history document |
| `/skills /skill:name` | List or use skills |
| `/mcp` | List MCP servers |
| `/permissions` | Show or change rules |
| `/undo /redo /retry` | Undo, redo, resend |
| `/rewind [k]` | Keep turns 1 to k |
| `/explain /review` | Prompt templates |
| `/exit` | Leave; return the session |

### WHILE AN ANSWER RUNS

In a terminal, type to steer. Ctrl-C pauses: **[s]**teer, **[f]**ollow-up, **[c]**ontinue, **[a]**bort, **[b]**ackground; RStudio and Positron use Esc. Approvals: **[y]**es, **[a]**lways, **[n]**o, **[?]**.

## System 1 and System 2




The model decides what `peter()` returns. A decision model (Jev, Clef) answers once per input; a conversational model works in turns.

### SYSTEM 1: TYPED DECISIONS

`rct = `**`peter`**`("Is this an RCT?", abstracts, model = jev)`
One logical answer per element, TRUE when P(yes) ≥ `threshold` (0.5).

**`gptr_prob`**`(rct)`
P(yes) per element; `"probabilities"` gives the matrix.

**`peter`**`("Which tissue?", cells, model = jev, choices = `**`c`**`("liver", "lung", "other"))`
One label per input: a character vector.

**`peter`**`("How severe?", events, model = jev, levels = `**`c`**`("low", "mid", "high"))`
A score: the expected zero-based level, such as 1.88.

**`peter`**`(q, x, model = jev, min_confidence = 0.8, uncertain = "stop")`
Abstain below a confidence: NA by default, or a fallback or an error.

```
if (isTRUE(ok)) message("Escalate.")
abstracts[rct]; sum(rct)
```
Answers are plain vectors for `if`, `for`, subsetting and `ifelse()`.

`s |> `**`peter`**`("Did it succeed?", model = jev)`
Judge a session's last answer; adds no turn.

```
m = if (isTRUE(hard)) "opus" else "haiku"
peter(task, model = m)
```
Route each task to a deep or a fast agent.

### SYSTEM 2: AGENTS

**`peter`**`("Plan it.", d, model = opus, .opts = `**`list`**`(thinking = "high"))`
More reasoning: `off` to `max`; or a suffix, `"anthropic/claude-sonnet-5-5:high"`.

## Models and providers

```
"anthropic/claude-sonnet-5-5:high"
```
provider · model id · thinking

Aliases work too: `sonnet`, `opus`, `haiku`, `gpt`, `flash`, `jev`, `claude_code`, `codex`.

| | |
|---|---|
| `anthropic/…` | Anthropic · `ANTHROPIC_API_KEY` |
| `openai/…` | OpenAI · `OPENAI_API_KEY` |
| `google/…` | Google Gemini · `GEMINI_API_KEY` |
| `openrouter/…` | Also groq, deepseek, mistral, together, xai, cerebras, fireworks · `<NAME>_API_KEY` |
| `azure/…` | Azure OpenAI · `AZURE_OPENAI_API_KEY`, `AZURE_OPENAI_ENDPOINT` |
| `bedrock/…` | AWS Bedrock · `AWS_BEARER_TOKEN_BEDROCK`, `AWS_REGION` |
| `ollama/…` | Ollama, local, no key; also lmstudio, llamacpp, vllm |
| `claude-cli` | Claude Code CLI · `claude auth login` |
| `codex` | Codex CLI · `codex login` |
| `jev` | System 1 · TypeSafe Jev · `TYPESAFE_API_KEY` |
| `ollama/clef` | System 1 · local Clef and Clef Flash, Ollama 0.35.1 or later |

```
gptr_models(provider = "ollama",
  refresh = TRUE)
```
List installed Ollama models, once per R process; for a cloud route, fetch its catalog.

```
m = list(id = "my-id", tool_call = TRUE)
gptr_config(providers = list(
  bedrock = list(models = list(m))))
```
Declare Bedrock or Azure models, then `model = "bedrock/my-id"`. Each call replaces `providers`.

```
gptr_register(gptr_provider("corp",
  "openai-completions", base_url = u,
  auth = "CORP_KEY", models = list(m)))
```
Any OpenAI-compatible endpoint. `api = "bedrock-converse"` signs Bedrock with SigV4.

```
eg = gptr_config()$egress
eg$openai = "ack"
gptr_config(egress = eg, .scope = "user")
```
Acknowledge sending context to a remote provider, for unattended runs.

## The script is the history

Bind a document, and each top-level `peter()` call gets a block with the R code the agent ran.

```
peter("Fit mpg on wt.", mtcars)
# >>> gptr:46d369 model=…
fit = lm(mpg ~ wt, mtcars)
coef(fit)[["wt"]]
#> [1] -5.344472
## Decision: linear fit
# <<< gptr:46d369
```
your prompt · block header · recorded code · output · note · end marker

```
gptr_doc("analysis.R")
```
Record calls into `.R`, `.Rmd`, `.qmd` or `.ipynb`.

```
gptr_source("analysis.R",
  replay = "replay")
```
Rerun: fresh blocks replay with no model call.

```
GPTR_REPLAY=replay Rscript analysis.R
```
Prove from the shell that a script needs no model.

```
gptr_blocks("analysis.R")
```
Each block's status: fresh, stale (prompt edited), user-edited or undone.

```
gptr_doc("report.Rmd")
rmarkdown::render("report.Rmd")
```
Render as usual; each block becomes a chunk named `gptr-<id>`.

```
gptr_doc("analysis.ipynb", sync = TRUE)
```
Apply blocks queued while it was open.

```
gptr_doc("log.R", format = "transcript")
```
Record the console as a runnable `.R` transcript.

```
peter(q, replay = "live")
```
Modes: `"auto"` (default), `"replay"`, `"live"`, `"record"`.

EXPORT A RECORDED .R

```
gptr_export("analysis.R", "plain.R")
```
Plain R: recorded code runs, `peter()` calls become comments.

```
gptr_export("analysis.R", "peter.R",
  format = "peter")
```
A `peter()` script: calls run, recorded code is commented.

## Try it offline

A scripted model runs every pattern with no key, server or network.

```
fake = gptr_fake_provider(list(
  list(tool = "r", input = list(
    code = "fit = lm(mpg ~ wt, mtcars)")),
  "Each 1000 lb lowers mpg by about 5.3."))
s = peter("Fit mpg on wt.", mtcars,
  model = fake, mode = auto)
s$text; coef(fit)
```
The first reply runs R; the second is the answer.

## Tools, files and artifacts

The model calls `read`, `r`, `edit`, `write` and `ask`. Inside `r` it uses the `peter$` helpers, which you can call too.

```
peter("Review R/.", tools = extended)
```
Presets: `minimal`, `standard`, `readonly`, `extended` (adds grep, find, ls).

```
peter(q, tools = c("-write", "+grep"))
```
Remove or add single tools.

```
peter("What is in data/scores.csv?")
```
The read tool returns text and images; binary files get a loading hint.

```
peter$read("R/fit.R", limit = 40)
peter$grep("lm\\(", path = "R")
peter$find("*.R"); peter$ls("R")
```
Read a file window, search contents, find and list files.

```
peter$edit("R/fit.R", list(list(
  oldText = "n = 10", newText = "n = 20")))
peter$write("notes.txt", "done\n")
```
Exact replacements; atomic writes.

```
peter$sh(c("git", "status"))
peter$bg(c("Rscript", "long.R"))
```
Run a program; run one in the background.

```
peter$sql("SELECT cyl, AVG(mpg) AS mpg
  FROM mtcars GROUP BY cyl", mtcars)
```
SQL on data frames (duckdb); `peter$py()` runs Python.

ARTIFACTS: SHINY APPS

```
peter("Make a Shiny app of it.", survey)
```
The agent writes `.gptr/artifacts/<id>/app.R` and launches it.

```
gptr_artifacts()
gptr_artifacts("survey-view", open = TRUE)
```
List versioned apps; open, stop or relaunch a version.

## Permissions and trust

| | |
|---|---|
| `plan` | Read only; R runs in a scratch environment |
| `manual` | Default: read-only actions run, others ask |
| `edits` | Also edit and write files in the project |
| `auto` | All but critical actions, gptr settings and secret reads |
| `yolo` | Nearly all; still asks before critical deletes |

```
gptr_permissions(deny = "r(sh:rm *)")
```
Rules: deny beats ask beats allow.

```
gptr_rewind(s)
```
Undo the last turn: objects, files and conversation.

The gate is advisory, not a sandbox; see `?gptr_security`.

## Skills, plugins and MCP

```
gptr_skills()
```
Skills from the project, user, plugins and packages.

```
peter("Speed up f.", f,
  skills = "high-performance-r")
```
Preload a skill; yours go in `.gptr/skills/<name>/SKILL.md`.

```
.gptr/prompts/check.md  # becomes /check
```
A prompt template is a console command.

```
peter("Report units.", plugins = "./lab")
```
Enable a plugin; `gptr_plugins()` lists them.

```
gptr_mcp_add("docs",
  url = "http://127.0.0.1:8000/mcp")
```
Add an MCP server (or `command =` for stdio).

```
peter$mcp$<server>$<tool>(...)
```
MCP tools are R functions the agent calls.

```
h = gptr_mcp_serve(); h$stop()
```
Serve this R session to Claude Code, Codex or Cursor.

```
up = gptr_tool("shout", "Upper-case text",
  fun = function(x) toupper(x))
off = gptr_register(up)
```
Add your own tool; `off()` removes it.

## Teams, fan-outs, background

```
peter("Review fit.", agents = list(
  stats = agent(system = "Check stats."),
  code = agent(system = "Check code.")))
```
A team of named roles; `s$stats$text` reads one.

```
f = peter("Summarise.", cohorts,
  parallel = 2)
```
One child per element, two at a time; each returns with `gptr_return()`.

```
gptr_parallel(a = peter("Plan."),
  b = peter("Review."))
```
Run independent calls concurrently.

```
b = peter("Check fit.", fit,
  background = TRUE)
gptr_wait(b)
```
Experimental: return at once; also `gptr_steer()`, `gptr_cancel()`, `gptr_jobs()`.

```
gptr_usage(s, by = "agent")
```
Tokens and cost by session, agent, model or route.

## Learn more

```
vignette("getting-started",
  package = "gptr")
```
Start here. Other guides: why-peter, language-models, system-one, interactive-console, configuration, script-as-history, tools-and-artifacts, teams-and-background, skills-and-plugins, mcp, extending-gptr, execution-model, token-efficiency.